

SPADE

A Superposition Physical-Layer AI Demodulation Dataset
(Source–Destination, USRP SDR)

Dataset Documentation

SyedMohammad Kashani · Ashfaq Khokhar · Sang Kim
Version 1.0.0 · September 2026

Contact: mkashani.phd@gmail.com
Repository: github.com/mkashani-phd/superposition-phy-security-dataset

1. Overview

Over-the-air measurements from a Source → Destination software-defined radio testbed for **SPADE** (Superposition Physical-layer AI DEmodulation): a dataset built for training and benchmarking **AI/ML-based joint demodulators** that recover a message and a superposed tag from a single physical-layer waveform. A second signal (the tag) is superposed directly onto the FSK-modulated message waveform at a fraction α of the message power, so the two are recovered jointly from the same received signal — a nontrivial detection problem well suited to data-driven, learned decoders.

Superposition-coded physical-layer tagging is a general mechanism with (at least) two security applications: **encryption** (the superposed tag carries cryptographic keystream material) and **authentication** (the superposed tag carries a verifiable message-authentication code). This release's primary focus is providing the raw demodulation data (soft-decision statistics, ground truth, and configuration) needed to train and evaluate AI decoders for both applications; the encryption/ part covers the encryption application, and the plain tag in `alpha_sweep/` corresponds to the authentication-style use case.

This release contains two dataset parts:

- **encryption/** — the encryption application: the superposed tag is an AES-CTR keystream chunk (not a plain HMAC), giving a cryptographically-generated, per-transmission physical-layer encryption tag. $\alpha = 0.5$, code rate $R = 0.5$.
- **alpha_sweep/** — a demodulation characterization dataset: the same superposition scheme with a fixed default message and a plain (non-cryptographic, authentication-style) tag, swept over $\alpha \in \{0.0, 0.1, 0.15, 0.2, 0.25, 0.3, 0.35, 0.4\}$ at $R = 0.5$, destination-only. This part exists to characterize how the superposition coding itself — and a demodulator's ability to separate message and tag — behaves across SNR and power allocation.

Beyond studying the coding scheme analytically, both parts are also structured as **machine-learning-ready datasets for data-driven decoding of superposition-coded signals**: raw per-symbol soft-decision statistics (`r0`, `r1`) are provided alongside ground-truth labels (`BER_tag`, `BER_msg`, and for encryption/ the exact AES-CTR keystream via `scripts/verify_keystream.py`), so a joint message+tag demodulator can be learned directly from data rather than hand-derived, across a range of SNR and α operating points.

2. Motivation

Wireless link security is typically added above the physical layer — e.g. encryption applied at the MAC layer or higher, with the PHY layer itself left as plaintext RF. SPADE instead explores adding security **at the physical layer** via superposition coding — superposing cryptographic (encryption) or verifiable (authentication) material directly onto the modulated waveform, independent of whatever security (if any) is applied at the MAC layer or above. This decouples physical-layer confidentiality/authenticity from MAC-layer support, so security is present on the air interface even when the MAC layer provides none, and layers can be composed without the PHY security mechanism depending on MAC-layer cooperation.

Jointly recovering the message and the superposed tag from one waveform is a nontrivial detection problem — the tag is designed to interfere minimally with the message, but the two are not perfectly separable, especially at higher α or lower SNR. This makes the problem a natural candidate for **AI/ML-based joint**

demodulation: instead of a hand-derived successive-cancellation or joint ML/MAP detector, a model can be trained directly on the raw r_0/r_1 soft-decision statistics in this dataset to learn the message/tag separation, using $\text{BER}_{\text{tag}}/\text{BER}_{\text{msg}}$ (and, for the encryption dataset, the exact reconstructed keystream) as ground truth.

3. Testbed

Two USRP software-defined radios (Source, Destination) placed in an office space at variable distance, communicating over the air at ~ 1.9 GHz using 2-level FSK modulation. The message (and, superposed on it, the tag) is preceded by a preamble/postamble for frame synchronization, then LDPC-coded at rate R , then FSK-modulated. The receiver records raw IQ samples for offline post-processing rather than decoding in real time.

For each received frame the two per-symbol soft-decision branches used for message/tag separation are stored as r_0 and r_1 (the two FSK tone-energy statistics the demodulator uses to compute LLRs / do joint or successive-cancellation detection).

4. Files & Dataset Structure

Note: the two .h5 files below (285 MB and 1.1 GB) exceed common per-file hosting limits (e.g. GitHub's 100 MB cap) and are distributed separately from the code repository (e.g. via IEEE DataPort). The code repository contains scripts and a tutorial notebook only.

Path	Size	Description
data/encryption/superposition_encryption_alpha0.5_R0.5.h5	285 MB	Primary encryption dataset ($\alpha = 0.5$, $R = 0.5$)
data/alpha_sweep/superposition_alpha_sweep_R0.5.h5	1.1 GB	Superposition coding characterization sweep ($R = 0.5$)
scripts/export_dataset.py	—	Provenance: how the files were generated from the source MongoDB (not needed to use the dataset)
scripts/verify_keystream.py	—	Regenerates the ground-truth AES-CTR tag for any encryption-dataset document
notebooks/dataset_tutorial.ipynb	—	Runnable walkthrough of everything in this document

4.1 encryption/superposition_encryption_alpha0.5_R0.5.h5

One top-level HDF5 group, destination, holding 1205 received frames.

4.2 alpha_sweep/superposition_alpha_sweep_R0.5.h5

One top-level group per superposition coefficient: $\alpha_{0.00}$, $\alpha_{0.10}$, $\alpha_{0.15}$, $\alpha_{0.20}$, $\alpha_{0.25}$, $\alpha_{0.30}$, $\alpha_{0.35}$, $\alpha_{0.40}$ (document counts range

from 190 to 6619 per group — these were recorded opportunistically, not with a fixed target count per alpha). Destination only.

4.3 Group layout (same in every group of both files)

<group>/documents/id	string	- MongoDB ObjectId of the source record (provenance only)
<group>/documents/SNR	float64	- measured SNR (dB) for this frame
<group>/documents/BER_tag	float64	- bit error rate of the recovered tag vs. ground truth
<group>/documents/BER_msg	float64	- bit error rate of the recovered message vs. ground truth
<group>/documents/sigma_n2	float64	- estimated noise variance
<group>/documents/config_json	string	- JSON-encoded experiment configuration
<group>/documents/time_stamp	string	- Unix timestamp string (encryption dataset only)
<group>/documents/enc_conf_json	string	- JSON-encoded encryption configuration (encryption only)
<group>/arrays/r0_data	float32	- flattened, concatenated r0 arrays
<group>/arrays/r0_lengths	int64	- length of each document's r0 array
<group>/arrays/r1_data	float32	- flattened, concatenated r1 arrays
<group>/arrays/r1_lengths	int64	- length of each document's r1 array

r0/r1 are variable-length per document in principle, so they are stored flattened with a parallel `_lengths` array giving each document's slice length — reconstruct with a cumulative sum of `_lengths` as offsets (the tutorial notebook has a `get_document()` helper that does this). In practice, within a given file all documents happen to share the same length (14976 for the encryption dataset, 14592 for the alpha sweep), but don't rely on that being true in general.

4.4 config_json fields (selected)

Field	Meaning
ALPHA	Superposition coefficient: fraction of power/amplitude given to the tag relative to the message
MSG_CODE_RATE	LDPC code rate R applied to the message
MAC_LDPC, MAC_REP	Code rate / repetition factor applied to the tag (non-encryption datasets)
MAC_SHA	Hash algorithm used to compute the plain HMAC tag (sha256) — not used in the encryption dataset, where the tag is an AES-CTR keystream instead
SOURCE, DESTINATION	USRP device identifiers (not IP addresses / no PII)
FREQ, RX_RATE, TX_RATE, TX_SPS	RF center frequency and sample rates
PREAMBLE, POSTAMBLE, PREAMBLE_REPEAT	Frame synchronization sequences
PAYLOAD	The fixed default test message transmitted in every frame (same across the whole dataset — not secret, it's placeholder text)
SUPERPOSED	Internal flag from the transmit code path used; ALPHA is the field that reflects the actual superposition weight applied

4.5 enc_conf_json fields (encryption dataset only)

Field	Meaning
KEY	AES-256 key (hex), used to generate the tag keystream — published as-is at the authors' request, see Section 6
IV	128-bit CTR initial counter value (hex)
COUNTER	1-based chunk index into the AES-CTR keystream used for this frame's tag (see caveat in Section 6)
PAYLOAD	Fixed default test payload used as the plaintext framing for this scheme
MSG_CODERATE	Code rate used for the message in this scheme

5. Quick Start

```
import h5py, json, numpy as np

f = h5py.File("data/encryption/superposition_encryption_alpha0.5_R0.5.h5", "r")
dest = f["destination"]

print(list(dest.keys()))          # ['arrays', 'documents']
print(dest.attrs["document_count"]) # 1205

lengths = dest["arrays/r0_lengths"][:]
offsets = np.concatenate([[0], np.cumsum(lengths)])
i = 0
r0_i = dest["arrays/r0_data"][offsets[i]:offsets[i+1]]
config_i = json.loads(dest["documents/config_json"][i])
```

See notebooks/dataset_tutorial.ipynb for a full runnable walkthrough (loading both files, building a metadata table with pandas, filtering by SNR, plotting BER vs. SNR across alpha).

6. Reconstructing the Ground-Truth Tag

In the encryption dataset, the tag isn't a fixed HMAC — it's a slice of an AES-CTR keystream generated from `enc_conf.KEY` and `enc_conf.IV`, taken at the `enc_conf.COUNTER`-th non-overlapping chunk of length equal to the message (each transmission advances the counter by one).

`scripts/verify_keystream.py` reimplements this slicing standalone (no dependency on the original lab codebase) so you can regenerate the exact bit sequence that was superposed as the tag for a given document, and use it as ground truth to score your own decoder's tag output directly — instead of only being able to check `BER_tag/BER_msg` after the fact.

```
pip install -r scripts/requirements.txt
python scripts/verify_keystream.py data/encryption/superposition_encryption_alpha0.5_R0.5.h5
```

Caveat: `enc_conf.COUNTER` is a snapshot of the shared counter value at the moment the receiver logged the frame, not a value that was individually verified against what the transmitter used for that exact frame. If transmitter and receiver ever drift out of sync (e.g. a dropped or corrupted frame between them), the reconstructed tag for a specific document could be off by one or more counter steps from what was actually transmitted. Treat exact per-document tag reconstruction as best-effort, and cross-check against `BER_tag/BER_msg` where available before relying on an exact match.

The KEY/IV are published unredacted (rather than being scrubbed for the release) so that users can independently verify this reconstruction procedure themselves, not only trust `scripts/verify_keystream.py`'s output.

7. Suggested Uses

- Training and benchmarking **AI/ML-based (e.g. deep learning) joint demodulators** for superposition-coded signals, using `r0/r1` as raw input features and `BER_tag/BER_msg` or the reconstructed ground-truth tag as labels, across a range of SNR and superposition-power (`alpha`) operating points (`alpha_sweep/`).
- Benchmarking classical joint message+tag demodulators (successive cancellation, joint ML/MAP detection) against the same ground truth, as a baseline for the above.
- Studying the BER/SNR trade-off of the two security applications of superposition tagging: **encryption** (a cryptographic AES-CTR tag, encryption/) versus **authentication** (a plain, verifiable tag, `alpha_sweep/` at `alpha=0`) on the same physical-layer waveform, keeping in mind the encryption dataset only covers `alpha=0.5`.

8. Known Limitations

- `alpha_sweep/` groups were collected opportunistically and are not balanced (190 to 6619 documents per `alpha` value) or matched in SNR range across `alpha` values.
- `BER_tag/BER_msg` in `alpha_sweep/` were computed by the original capture pipeline's own decoder (a specific baseline demodulator), not necessarily an optimal joint or successive-cancellation detector. Treat them as a baseline to beat rather than a ceiling — recompute your own from `r0/r1` with your detector to compare fairly.
- See the ground-truth tag caveat in Section 6 regarding TX/RX counter synchronization.

9. Provenance

Originally recorded into MongoDB by the lab's experiment-control code and exported to HDF5 with `scripts/export_dataset.py` (kept in the code repository for transparency; you do not need MongoDB or that script to use this dataset — the `.h5` files are self-contained).

10. Citation

If you use this dataset, please cite it as:

Kashani, S., Khokhar, A., Kim, S. (2026).
"SPADE: A Superposition Physical-Layer AI Demodulation Dataset
(Source-Destination, USRP SDR)", v1.0.0.
<https://github.com/mkashani-phd/superposition-phy-security-dataset>

A machine-readable citation is also provided as `CITATION.cff` in the code repository.

11. License

CC BY 4.0 — attribution required for reuse.

12. Contact

mkashani.phd@gmail.com